

Tre modi di passare un file

FTP, NFS e SMB, ognuno sul suo server, tutti raggiunti da un unico client sullo stesso cavo. Stesso compito, tre risposte — e tre idee diverse di chi sei. Da un lab in esecuzione.

Comando `qlab run filesharing-lab`

VM `filesharing-lab-ftp · -nfs · -samba · -client`

LAN `192.168.100.0/24`, isolata fra le quattro VM

Credenziali `labuser / labpass` · utenti di lab `alice`, `bob` / `labpass`

Risultato `qlab test filesharing-lab` → 4 esercizi, 52 controlli, tutti superati

1. Quattro macchine, un cavo

filesharing-lab-ftp	192.168.100.1/24
filesharing-lab-nfs	192.168.100.2/24
filesharing-lab-samba	192.168.100.3/24
filesharing-lab-client	192.168.100.10/24

Tre server, un protocollo ciascuno, e un client che parla con tutti. Gli indirizzi sono fissi, quindi niente in questa guida dipende da cosa ha deciso il DHCP. La macchina interessante è il client: tutto quello che viene dopo è lo stesso compito — leggere un file che sta altrove — svolto tre volte da lì.

Separare i server è il punto. Su una sola macchina i tre protocolli si confondono; su tre macchine ognuno deve portarsi il proprio trasporto, la propria autenticazione e la propria idea di cosa sia un "file".

2. Cosa sta girando davvero su ogni server

```
--- ftp (vsftpd) ---
active
anonymous_enable=YES
anon_root=/srv/ftp/shared
local_enable=YES
write_enable=YES
chroot_local_user=YES
pasv_enable=YES
--- nfs (NFS-Ganesha: an NFS server that runs in userspace) ---
active
  Protocols = 3, 4;
  Path = /srv/nfs/shared;
  Pseudo = /shared;
  Access_Type = RW;
  Squash = No_Root_Squash;
    Clients = 192.168.100.0/24;
    Access_Type = RW;
  Path = /srv/nfs/readonly;
  Pseudo = /readonly;
  Access_Type = RO;
  Squash = Root_Squash;
    Clients = 192.168.100.0/24;
    Access_Type = RO;
--- samba (smbd) ---
active
[global]
[shared]
  path = /srv/samba/shared
  read only = No
  valid users = alice bob
[public]
  guest ok = Yes
  path = /srv/samba/public
```

Tre demoni, tre stili di configurazione.

vsftpd si configura con righe piatte `chiave=valore`. Due contano qui: `anonymous_enable` apre un login senza account dietro, con radice in `anon_root`, e `chroot_local_user` chiude un account vero dentro la propria home. Un solo demone, due esperienze molto diverse a seconda del nome che digiti.

Il server NFS è **NFS-Ganesha**, che gira in user space e non nel kernel — vale la pena notarlo, perché quasi tutta la documentazione NFS dà per scontato `nfs-kernel-server` e `/etc/exports`, e qui non esiste né l'uno né l'altro. Ogni export dichiara un `Path` reale, un nome `Pseudo` che i client usano al suo posto, un `Access_Type` e una politica di `Squash`. I due export differiscono esattamente in questi ultimi due campi.

Samba risponde a `testparm` con la configurazione che ha effettivamente interpretato. `shared` ha `valid users`, `public` ha `guest ok` — la stessa distinzione che `vsftpd` fa con `anonymous_enable`, scritta in un altro dialetto.

3. FTP: una conversazione con i numeri davanti

```
$ a whole FTP session, scripted – anonymous first
Connected to 192.168.100.1.
220 (vsFTPd 3.0.5)
230 Login successful.
Remote system type is UNIX.
229 Entering Extended Passive Mode (|||30037|)
150 Here comes the directory listing.
-r-xr-xr-x   1 109      117          43 Sep 14 13:22 README.txt
-r-xr-xr-x   1 109      117          46 Sep 14 13:22 sample.txt
226 Directory send OK.
229 Entering Extended Passive Mode (|||30061|)
150 Opening BINARY mode data connection for README.txt (43 bytes).
226 Transfer complete.
221 Goodbye.

$ and the same server as a real account
220 (vsFTPd 3.0.5)
230 Login successful.
Remote system type is UNIX.
Remote directory: /
221 Goodbye.

-rw-rw-r-- 1 labuser labuser 43 Sep 14 13:22 /tmp/from-ftp.txt
Welcome to the filesharing-lab FTP server.
```

FTP è abbastanza vecchio da essere leggibile. Ogni risposta del server è un codice di tre cifre più una frase: `220` alla connessione, `230` per un login riuscito, `150` prima che i dati comincino a muoversi, `226` quando hanno finito, `221` all'uscita. Il protocollo è tutto qui, ed è il motivo per cui FTP è quello che si impara a debuggare a mano con `telnet`.

Qui girano due login contro lo stesso demone. Quello anonimo atterra nella directory indicata da `anon_root` e vede i due file che ci sono stati messi. Poi entra `alice` con una password vera e `pwd` risponde `/` — non perché sia nella radice del server, ma perché `chroot_local_user` ha fatto *sembrare* radice la sua home. Della directory anonima, e di tutto il resto della macchina, non vede nulla.

La password è passata in chiaro. Come tutto il resto dell'elenco. FTP è precedente all'idea che la rete sia ostile; su una rete vera qui si usa FTPS o SFTP, e il lab tiene FTP in chiaro proprio perché così lo si può leggere.

4. NFS: il file server che sparisce

```
$ showmount -e 192.168.100.2      (what the server offers)
Export list for 192.168.100.2:
/srv/nfs/shared    192.168.100.0/24
/srv/nfs/readonly  192.168.100.0/24

$ mount -t nfs, then look at what landed in the filesystem
192.168.100.2:/shared on /mnt/nfs type nfs4
(rw,relatime,vers=4.2,rsize=1048576,wsiz=1048576,namlen=255,hard,proto=tcp,timeo=600,retrans=2,sec=sys,
clientaddr=192.168.100.10,local_lock=none,addr=192.168.100.2)

total 4
-rw-r--r-- 1 alice alice 37 Sep 14 13:22 README.txt

$ writing through the mount
-rw-r--r-- 1 root root 36 Sep 14 13:35 /mnt/nfs/from-client.txt
```

Qui niente sembra un protocollo di rete, ed è esattamente l'idea. Dopo `mount`, la directory remota è una directory. `ls` funziona. Scrivere funziona. Nessun programma client, nessuna sessione, nessun comando di trasferimento: il kernel traduce normali chiamate di filesystem in richieste NFS e viceversa.

`showmount -e` chiede al server cosa offre prima di impegnarsi in qualcosa, e risponde con i percorsi reali. Il mount invece usa il percorso **pseudo**: NFSv4 appende tutti gli export a un'unica radice virtuale, così i client chiedono `/shared` e non sanno mai dove stia davvero sul disco del server.

La riga di `mount` merita una lettura. `vers=4.2` è la versione negoziata, `hard` significa che una richiesta riprova all'infinito invece di fallire se il server è irraggiungibile per un attimo, e `sec=sys` è l'autenticazione — su cui torniamo subito.

5. `sec=sys` : l'identità è un numero che spedisce

```
NFS moves numbers, not names. The same file, from both ends:
--- on the server ---
total 8
-rw-r--r-- 1 alice alice 37 Sep 14 13:22 README.txt
-rw-r--r-- 1 root  root  36 Sep 14 13:35 from-client.txt

total 8
-rw-r--r-- 1 2001 2001 37 Sep 14 13:22 README.txt
-rw-r--r-- 1 0    0    36 Sep 14 13:35 from-client.txt
uid=2001(alice) gid=2001(alice) groups=2001(alice)
--- on the client ---
total 8
-rw-r--r-- 1 2001 2001 37 Sep 14 13:22 README.txt
-rw-r--r-- 1 0    0    36 Sep 14 13:35 from-client.txt
```

```
total 8
-rw-r--r-- 1 alice alice 37 Sep 14 13:22 README.txt
-rw-r--r-- 1 root  root  36 Sep 14 13:35 from-client.txt
uid=2001(alice) gid=2001(alice) groups=2001(alice)
```

Gli stessi due file, elencati su entrambe le macchine, prima come numeri e poi come nomi.

Sul cavo passano i numeri; i nomi non passano mai. `alice` è uid 2001 sul server, e questo lab dà al client una `alice` anch'essa a uid 2001, quindi le due parti stampano lo stesso nome e sembra tutto naturale. Cambia l'uid di `alice` sul client in 1001 e il file risulterà di chiunque *sia* 2001 lì — o di un nudo `2001` se non c'è nessuno. Niente si romperebbe, e niente ti avviserebbe.

`from-client.txt` è l'altra metà della lezione. È stato scritto con `sudo` dal client e sul server appartiene a `root`, perché questo export è configurato `Squash = No_Root_Squash`: il server prende l'uid 0 per buono. Il default — ed è quello che usa l'export in sola lettura — è `Root_Squash`, che mappa un root remoto su `nobody` proprio perché l'accesso amministrativo su un client non diventi accesso amministrativo ai file del server.

`sec=sys` vuol dire che il server si fida di quello che il client dichiara su chi sta chiedendo. È un modello del tutto ragionevole su una rete di sala macchine che controlli, ed è il motivo per cui NFS normalmente si tiene fuori da qualsiasi rete che non controlli. L'alternativa, `sec=krb5`, mette Kerberos nel percorso: l'identità si dimostra invece di dichiararla.

6. SMB: condivisioni, non filesystem

```
$ smbclient -L (what the server publishes)

Sharename      Type      Comment
-----
shared         Disk      Authenticated Read-Write Share
public         Disk      Public Read-Only Share
IPC$           IPC       IPC Service (filesharing-lab Samba Server)
SMB1 disabled -- no workgroup available

$ fetch a file over SMB
getting file \README.txt of size 39 as from-smb.txt (390000.0 KiloBytes/sec) (average inf KiloBytes/sec)
.              D          0   Mon Sep 14 13:22:35 2026
..             D          0   Mon Sep 14 13:22:35 2026
README.txt     N          39  Mon Sep 14 13:22:35 2026

2051980 blocks of size 1024. 465716 blocks available
-rw-r--r-- 1 labuser labuser 39 Sep 14 13:35 /tmp/from-smb.txt

$ the same share without credentials
tree connect failed: NT_STATUS_ACCESS_DENIED
```

SMB sta in mezzo agli altri due. È un protocollo di sessione come FTP — ti colleghi, ti autentichi, dai comandi — ma quello che espone è una *share*, un albero con un nome che si può montare come NFS o sfogliare come FTP. `smbclient` qui è la versione a mano.

L'elenco delle share è il catalogo pubblico del server: le due share configurate più `IPC$`, il canale di controllo che SMB usa per l'elenco stesso. Poi `get` tira giù un file. La riga "SMB1

disabled" non è un errore: è `smbclient` che segnala che il vecchio protocollo di browsing dei workgroup è spento, come deve essere.

L'ultimo comando è quello che conta: stessa share, senza credenziali, `NT_STATUS_ACCESS_DENIED`. L'autorizzazione è per share e la verifica il server, ed è la differenza strutturale con NFS. Qui decide il server; con NFS `sec=sys` decide il client.

7. I tre a confronto

Same job, three protocols, seen from one machine:

PROTO	PORT	HOW IT APPEARS	IDENTITY
FTP	21 + data	a session; you copy files in and out	username + password, in clear
NFS	2049	a directory in your filesystem	numeric uid/gid, trusted
SMB	445	a share you mount or browse	username + password, per share

```
$ what is actually mounted on this client right now
192.168.100.2:/shared on /mnt/nfs type nfs4 (rw, relatime, vers=4.2, rsize=1048576, wsize=1048576, n
```

```
$ and the three servers, from the client
192.168.100.1:21 open
192.168.100.2:2049 open
192.168.100.3:445 open
```

I tre server rispondono sulle tre porte, e in `mount` compare esattamente uno dei tre protocolli — perché solo NFS è stato progettato per diventare parte del filesystem locale. Anche FTP e SMB si possono far sembrare tali (`curlftpfs`, `mount -t cifs`), ma per FTP è un'illusione tutta lato client, e per SMB è un secondo modo di usare un protocollo che una sessione ce l'aveva già.

Il riassunto utile è la colonna dell'identità. FTP nomina un utente per sessione, in chiaro. SMB nomina un utente per share, verificato dal server. NFS non nomina nessuno: manda un numero e si fida. Tre protocolli, tre risposte all'unica domanda che davvero li separa.

8. Provaci

```
qlab run filesharing-lab
qlab shell filesharing-lab-client
```

Dal client:

```
ftp -inv 192.168.100.1 # anonymous / alice+labpass
sudo mount -t nfs 192.168.100.2:/shared /mnt/nfs-shared
sudo mount -t nfs 192.168.100.2:/readonly /mnt/nfs-readonly
smbclient //192.168.100.3/shared -U alice%labpass -m SMB3
smbclient //192.168.100.3/public -N -m SMB3
```

Tre cose che vale la pena fare dopo:

- Scrivere in `/mnt/nfs-readonly` e guardare quale livello ti dice di no.
- Cambiare l'uid di `alice` sul client, rimontare, e vedere cambiare il proprietario di `README.txt` senza un solo messaggio di errore.

- Lanciare `sudo tail -f /var/log/vsftpd.log` sul server FTP mentre ti colleghi dal client: tutta la conversazione, da entrambe le parti, in chiaro.

```
Exercise 1 – FTP with vsftpd
  Exercise 1: All 11 checks passed
Exercise 2 – NFS with Ganesha
  Exercise 2: All 12 checks passed
Exercise 3 – Samba
  Exercise 3: All 14 checks passed
Exercise 4 – Cross-Protocol Comparison
  Exercise 4: All 15 checks passed
All 4 exercise(s) passed!
```

`qlab test filesharing-lab` esegue 52 controlli sulle quattro VM. L'ultimo esercizio è quello trasversale, e il suo controllo `Alice UID matches (2001)` c'è per il motivo che spiega la sezione 5: in questo lab il numero coincide, e il test lo dice ad alta voce invece di lasciar credere alla magia.